

## Buổi 1 — Architectural Thinking & Trade-offs

**Chủ đề:** Architectural characteristics, trade-offs, constraints

### **Mục tiêu**

- Hiểu tư duy kiến trúc và cách đưa ra quyết định dựa trên trade-off, chứ không phải “best practice”.

### **Bài tập thực hành**

Cho một hệ thống “E-commerce mini”, hãy:

1. Xác định 7–10 architectural characteristics (availability, elasticity, performance, modifiability...).
2. Sắp xếp ưu tiên đặc tính bằng kỹ thuật **utility tree**.
3. Viết 5 quyết định kiến trúc có trade-off (ví dụ: SQL vs NoSQL, sync vs async).

### **Deliverable:**

- Utility tree
- Trade-off analysis
- Architectural decision record (ADR)

## ARCHITECTURAL CHARACTERISTICS (*QUALITY ATTRIBUTES*)

**Architectural characteristics** (*quality attributes*) là những **thuộc tính phi chức năng** quyết định **hình dạng kiến trúc**, chứ không phải tính năng:

1. Availability: tính sẵn sàng
2. Performance: hiệu năng
3. Scalability: khả năng mở rộng
4. Elasticity: co giãn tài nguyên
5. Consistency: tính nhất quán dữ liệu
6. Reliability: độ tin cậy
7. Security: bảo mật
8. Modifiability: dễ thay đổi
9. Maintainability: dễ bảo trì
10. Observability: khả năng quan sát (log lỗi dễ tìm hay không?, điều tra bug có nhanh hay không?)
11. Testability: khả năng kiểm thử
12. Cost Efficiency: hiệu quả chi phí

## UTILITY TREE (ATAM – ARCHITECTURE TRADEOFF ANALYSIS METHOD)

### 1. Utility Tree là gì?

Utility Tree là một cấu trúc phân cấp dùng trong ATAM, nhằm biểu diễn và ưu tiên các đặc tính kiến trúc (Architecture Characteristics / Quality Attributes) của hệ thống thông qua các kịch bản chất lượng (Quality Attribute Scenarios) làm cơ sở cho phân tích đánh đổi (tradeoff) và ra quyết định kiến trúc.

### 2. Mục đích:

1. Chuyển các yêu cầu chất lượng mơ hồ → thành kịch bản cụ thể, đo được
2. Ưu tiên các đặc tính kiến trúc (High – H / Medium – M / Low - L)
3. Làm cơ sở chính cho tradeoff của kiến trúc
4. Biện minh có các quyết định kiến trúc (Architectural decision record – ADR)

### 3. Cấu trúc:

#### Utility (Root)

|\_\_\_ **Architecture Characteristic** – Đặc tính kiến trúc (Availability, Performance , ...)

|\_\_\_ **Attribute Refinement** – Tinh chỉnh thuộc tính (Response Time, ...)

|\_\_\_ **Quality Attribute Scenario** – Kịch bản chất lượng

- **Stimulus** – Tác nhân

- **Environment** – Môi trường

- **Response** – Phản hồi của hệ thống

- **Response Measure** – Thước đo phản hồi

|\_\_\_ **Priority** – Độ ưu tiên

- **Importance (High / Medium / Low)** – Mức độ quan trọng

- **Difficulty (High / Medium / Low)** – Độ khó thực hiện

4. Utility Tree không giúp chọn kiến trúc “**Tốt nhất**” mà giúp chọn kiến trúc “**phù hợp với bối cảnh (context)**”

## 5. *Architectural decision record (ADR) template*

**ADR-XXX: <Tiêu đề quyết định>**

1. **Status (Trạng thái):** Proposed (Đề xuất) / Accepted (Chấp nhận) / Rejected (Từ chối) / Deprecated (Đã lỗi thời)
2. **Context (Ngữ cảnh):** mô tả bối cảnh kiến trúc dẫn đến quyết định
  - Hệ thống đang giải quyết vấn đề gì?
  - Các yêu cầu nghiệp vụ liên quan
  - Các architectural characteristics bị ảnh hưởng (performance, scalability, availability, consistency, ...)
  - Các ràng buộc (constraints):
    - Thời gian
    - Nhân lực
    - Công nghệ
    - Ngân sách, ...
  - Problem Statement (Bài toán cần quyết định): mô tả vấn đề kiến trúc cụ thể  
“Quyết định <cái gì> để đạt được <mục tiêu> trong bối cảnh <điều kiện>.”  
Ví dụ:
    - Chọn kiểu kiến trúc nào?
    - Giao tiếp synchronous hay asynchronous?
    - SQL hay NoSQL?
    - Saga orchestration (Mediator Topology) hay choreography (Broker Topology)?
3. **Decision Drivers (Yếu tố dẫn dắt quyết định)**  
Danh sách các yếu tố quan trọng nhất ảnh hưởng đến quyết định:
  - Performance (High)
  - Availability (High)
  - Modifiability (Medium)
  - Cost efficiency (Medium)
  - Team experience (High)
4. **Considered Options (Các phương án đã xem xét)**
  - Option 1: ...
  - Option 2: ...
5. **Decision (Quyết định)**  
Quyết định chọn Option X?

**6. Giải thích vì sao chọn quyết định này:**

- Lợi ích chính?
- Những đánh đổi phải chấp nhận?
- Quality attribute nào được ưu tiên, cái nào bị hy sinh?

**7. Consequences (Hệ quả)**

- Lợi ích
- Rủi ro

Cho một hệ thống “E-commerce mini”, hãy:

1. Xác định 7–10 architectural characteristics (availability, elasticity, performance, modifiability...).
2. Sắp xếp ưu tiên đặc tính bằng kỹ thuật **utility tree**.

Viết 5 quyết định kiến trúc có trade-off (ví dụ: SQL vs NoSQL, sync vs async).

### 1. Giả định bối cảnh 01 (boundary context):

#### a. Mô tả hệ thống:

- Website bán hàng online quy mô nhỏ – trung bình
- 5.000 ~ 10.000 users/ngày
- Chức năng chính
  - Xem sản phẩm
  - Đặt hàng
  - Thanh toán
- Có thể mở rộng trong tương lai

#### b. Ràng buộc:

- Team dev 3~5 members
- Ngân sách hạn chế
- Thời gian phát triển ngắn (3 tháng)

→ Chọn **Modular Monolithic** ko chọn **Monolithic, Microservice**

2. Xác định 7–10 architectural characteristics (availability, elasticity, performance, modifiability...).

| STT | Architectural Characteristic | Lý do                                                                                                                                                                                                                                                                                                                                             |
|-----|------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1   | <b>Availability (H)</b>      | Availability: là khả năng hệ thống <b>hoạt động liên tục và sẵn sàng phục vụ người dùng</b> .<br>→ “Người dùng có vào đặt hàng được hay không?”<br>→ Không đặt được hàng = mất doanh thu trực tiếp                                                                                                                                                |
| 2   | <b>Performance (H)</b>       | Performance là <b>tốc độ phản hồi</b> của hệ thống.<br>Không chỉ là “nhanh”: <ul style="list-style-type: none"><li>• Response time: User chờ bao lâu để có kết quả (ms)</li><li>• Throughput: Hệ thống xử lý được bao nhiêu (req/s)</li><li>• Latency: Mất bao lâu cho 1 yêu cầu đi–về (ms)</li></ul><br>→ Checkout chậm → người dùng bỏ giỏ hàng |
| 3   | <b>Scalability</b>           | Có khả năng user tăng khi có Flash sale                                                                                                                                                                                                                                                                                                           |
| 4   | <b>Elasticity</b>            | Scale lên/xuống để tiết kiệm chi phí                                                                                                                                                                                                                                                                                                              |
| 5   | <b>Consistency</b>           | Tránh bán vượt tồn kho                                                                                                                                                                                                                                                                                                                            |
| 6   | <b>Modifiability</b>         | Dễ thêm payment, shipping, promotion                                                                                                                                                                                                                                                                                                              |
| 7   | <b>Security</b>              | Thanh toán, dữ liệu cá nhân                                                                                                                                                                                                                                                                                                                       |

|   |                        |                              |
|---|------------------------|------------------------------|
| 8 | <b>Observability</b>   | Debug lỗi, theo dõi đơn hàng |
| 9 | <b>Cost Efficiency</b> | Ngân sách hạn chế            |

### 3. Sắp xếp ưu tiên đặc tính bằng kỹ thuật **utility tree**

#### **Utility (Giá trị tổng thể của hệ thống)**

Xây dựng hệ thống **E-commerce mini** đáp ứng nhu cầu kinh doanh hiện tại, **ổn định – nhanh – dễ mở rộng**, phù hợp với **team nhỏ, ngân sách hạn chế**.

#### **a. Performance (Hiệu năng) – High**

**Refinement:** Response Time

- **P1:** Người dùng duyệt sản phẩm, trang tải < **2 giây** trong điều kiện tải bình thường  
(*Importance: High – Difficulty: Medium*)
- **P2:** Thao tác **checkout & thanh toán** phản hồi < **1 giây** khi hệ thống không quá tải  
(*Importance: High – Difficulty: Medium*)

#### **b. Availability (Tính sẵn sàng) – High**

**Refinement:** Uptime, Fault isolation

- **A1:** Hệ thống đạt **99,5% uptime**, downtime không quá 3–4 giờ/tháng  
(*Importance: High – Difficulty: Medium*)
- **A2:** Lỗi ở thanh toán **không làm sập toàn hệ thống**  
(*Importance: High – Difficulty: Medium*)

#### **c. Consistency (Tính nhất quán dữ liệu) – High**

**Refinement:** Data consistency

- **C1:** Không có đơn hàng bị **mất hoặc tạo trùng**  
(*Importance: High – Difficulty: Medium*)
- **C2:** Không bán vượt tồn kho sau khi hoàn tất thanh toán  
(*Importance: High – Difficulty: High*)

#### **d. Modifiability (Khả năng thay đổi) – Medium**

**Refinement:** Ease of change

- **M1:** Thêm phương thức thanh toán mới ≤ **3 ngày**, không ảnh hưởng luồng đặt hàng  
(*Importance: Medium – Difficulty: Medium*)
- **M2:** Thay đổi rule khuyến mãi **không cần refactor lớn**  
(*Importance: Medium – Difficulty: Medium*)

#### **e. Scalability (Khả năng mở rộng) – Medium**

**Refinement:** Load growth

- **S1:** Hệ thống xử lý **lưu lượng tăng gấp 3 lần** trong đợt khuyến mãi  
(*Importance: Medium – Difficulty: Medium*)

**f. Elasticity (Khả năng co giãn) – Low**

**Refinement:** Auto scaling

- **E1:** Backend có thể scale theo tải, nhưng **không yêu cầu real-time auto scale**  
(*Importance: Low – Difficulty: Low*)

**g. Security (Bảo mật) – High**

**Refinement:** Confidentiality, Integrity

- **SE1:** Dữ liệu người dùng & thanh toán được mã hóa, không rò rỉ  
(*Importance: High – Difficulty: Medium*)

**h. Simplicity (Tính đơn giản) – High**

**Refinement:** Maintainability

- **SI1:** Kiến trúc **đễ hiểu – dễ bảo trì** cho team 3–5 dev  
(*Importance: High – Difficulty: Low*)

➔ Ưu tiên **Performance, Availability, Consistency, Simplicity** do hệ thống nhỏ và thời gian gấp

➔ Chấp nhận **Elasticity thấp** để giảm chi phí & độ phức tạp

➔ Utility Tree này **phù hợp kiến trúc Modular Monolith**, có thể tiến hóa lên Microservices sau.

4. Viết 5 quyết định kiến trúc có trade-off (ví dụ: SQL vs NoSQL, sync vs async)

**a. ADR-01: chọn kiến trúc Modular Monolith thay vì Microservices**

- **Status:** Accepted
- **Context:** Hệ thống E-commerce mini phục vụ 5.000–10.000 users/ngày, chức năng chính gồm xem sản phẩm, đặt hàng, thanh toán.
  - Các architectural characteristics ưu tiên cao từ Utility Tree:
    - Simplicity (High)
    - Performance (High)
    - Availability (High)
    - Modifiability (Medium)
  - Ràng buộc:
    - Team nhỏ
    - Thời gian phát triển ngắn

- Ngân sách hạn chế
  - **Problem Statement:** nên chọn **Monolithic**, **Modular Monolith** hay **Microservices** để cân bằng giữa đơn giản và khả năng mở rộng?
  - **Decision Drivers**
    - Simplicity
    - Time-to-market: khoảng thời gian từ khi một ý tưởng sản phẩm được hình thành cho đến khi sản phẩm đó được phát hành rộng rãi ra thị trường.
    - Team size
    - Maintainability
  - **Decision:** Chọn **Modular Monolith**.
  - **Consequences (hệ quả):**
    - **Ưu điểm:**
      - Codebase đơn giản, dễ debug
      - Triển khai nhanh
      - Hiệu năng tốt do không có network latency nội bộ
    - **Nhược điểm:**
      - Scale theo toàn hệ thống
      - Khi hệ thống lớn hơn, cần refactor sang microservices
- ➔ Modular Monolith là **bước đệm kiến trúc hợp lý** cho E-commerce mini trước khi tiến hóa

**b. ADR-02: Sử dụng Database SQL thay vì NoSQL cho Order & Payment**

- **Status:** Accepted
- **Context**
  - Các đặc tính ưu tiên cao:
    - **Consistency (High)**
    - **Availability (High)**
    - **Security (High)**
  - Đơn hàng và thanh toán yêu cầu dữ liệu chính xác, không trùng lặp, không mất.
- **Problem Statement:** nên dùng **SQL** hay **NoSQL** cho dữ liệu nghiệp vụ chính?
- **Decision Drivers**
  - Strong consistency
  - Transaction support
  - Data integrity



- **Decision:** chọn SQL (PostgreSQL/MySQL) cho Order, Payment.

- **Consequences**

- **Ưu điểm:**

- ACID transaction
    - Dễ đảm bảo consistency
    - Phù hợp nghiệp vụ tài chính

- **Nhược điểm:**

- Scale ngang khó hơn NoSQL
    - Schema cứng hơn

➔ Consistency quan trọng hơn scalability trong giai đoạn đầu.

c. **ADR-03: Giao tiếp Sync cho User-facing (hướng người dùng), Async cho nghiệp vụ nền (background logic)**

- **Status:** Accepted

- **Context:**

- Từ Utility Tree:

- **Performance (High)**
    - **Availability (High)**

➔ User cần phản hồi nhanh khi đặt hàng, nhưng các xử lý nền có thể bất đồng bộ.

- **Problem Statement:** dùng sync hay async cho giao tiếp giữa các thành phần?

- **Decision Drivers:**

- Response time
  - User experience
  - Fault isolation

- **Decision**

- **Sync (HTTP/REST)** cho frontend → backend
  - **Async (event/message)** cho xử lý nghiệp vụ background

- **Consequences**

- **Ưu điểm:**

- Trải nghiệm người dùng tốt
    - Giảm coupling giữa các module

- **Nhược điểm:**

- Debug async phức tạp hơn
    - Cần cơ chế retry & monitoring

→ Kết hợp sync + async là trade-off hợp lý giữa UX và độ ổn định.

d. ADR-04: Sử dụng Eventual Consistency cho Inventory

▪ **Consequences**

- **Ưu điểm:**
  - Hệ thống resilient hơn
  - Không block user khi có sự cố nhỏ
- **Nhược điểm:**
  - Có thể có độ trễ cập nhật tồn kho
  - Cần xử lý bù (compensation)

\* **Strong Consistency (nhất quán mạnh = chính xác tuyệt đối):**

- Ghi xong → đọc ngay → thấy dữ liệu mới nhất
- Ví dụ: chuyển tiền ngân hàng → xử lý toàn bộ chuyển tiền trong **1 transaction ACID**

**Flow:**

1. User gửi yêu cầu chuyển tiền (A → B)
2. Bắt đầu transaction
3. Lock tài khoản A và B
  - SELECT ... FOR UPDATE
  - Không ai có thể sửa số dư A và B trong lúc này
4. Kiểm tra số dư tài khoản A
  - Nếu không đủ tiền → ROLLBACK
  - Nếu đủ → tiếp tục
5. Trừ tiền tài khoản A
6. Cộng tiền tài khoản B
7. Commit
  - A đã bị trừ tiền
  - B đã được cộng tiền
  - Log giao dịch được ghi

\* **Eventual Consistency (nhất quán cuối cùng):**

- Ghi xong → có thể đọc thấy dữ liệu cũ trong vài mili-giây / giây
- Sau một thời gian → mọi nơi sẽ đồng bộ

VD: **bối cảnh Microservices:**

Order Service

→ gửi event "OrderCreated"

→ Inventory Service cập nhật tồn kho

**Flow:**

1. User đặt hàng
2. Order Service tạo đơn thành công
3. Gửi event sang Inventory
4. Inventory cập nhật stock

➔ Trong khoảng thời gian giữa bước 2 và 4:

- Tồn kho có thể **chưa giảm**
- Hệ thống đang ở trạng thái **temporarily inconsistent (tạm thời không nhất quán)**

*Nhưng:*

- Sau khi event được xử lý
- Stock sẽ đúng

\* Một hệ thống **resilient** (là khả năng của hệ thống duy trì được mức độ phục vụ chấp nhận được ngay cả khi có lỗi xảy ra):

- **1 service chết** → hệ thống **không sập toàn bộ**
- **1 request lỗi** → **retry / fallback**
- **1 dependency timeout** → **degrade gracefully: che giấu lỗi**
- **Lỗi tạm thời** → **tự phục hồi**

e. ADR-05: Không dùng Saga phức tạp, xử lý nghiệp vụ bằng transaction nội bộ

...

**Giả định bối cảnh 02 (boundary context):**

- a. Mô tả hệ thống:
    - Website bán hàng online
    - 50.000 – 100.000 users/ngày (tăng trưởng nhanh)
    - Có flash sale định kỳ
    - Có mobile app + web
    - Tích hợp nhiều hệ thống bên ngoài:
      - Payment Gateway
      - Shipping provider
      - Recommendation engine
    - Có kế hoạch mở rộng quốc tế
  - b. Ràng buộc
    - Team mở rộng lên 12–20 developers
    - Có DevOps riêng
    - Hạ tầng cloud (AWS/GCP/Azure)
    - Ngân sách trung bình
    - Yêu cầu uptime cao
1. Xác định 7–10 architectural characteristics (availability, elasticity, performance, modifiability...).
  2. Sắp xếp ưu tiên đặc tính bằng kỹ thuật **utility tree**.
- Viết 5 quyết định kiến trúc có trade-off